Enlight Lab

Client Demo Explained

Plain-language guide for every button in the Live Demo.
Written for presenters - share only http://localhost:30900 with clients.
Project: D:\enlight-lab-platform

How to use this guide

Each section follows the same pattern: what the demo proves, what happens when you click, what the client sees, and a short script you can say out loud.

Before the client joins (hide terminal)

```
cd D:\enlight-lab-platform
.\scripts\go-live.bat
.\start-demo-control.bat
Open http://localhost:30900 and click Refresh
```

One URL for the client

- Live Demo: http://localhost:30900 (only screen to share)
- Optional tabs for you: Deployments :8082, Build pipelines on GitHub
- Do NOT open :4566 - that is background plumbing, not a website

Demo order on screen (top to bottom)

1. **Release safety**
2. **Outage response**
3. **Cloud config guard**
4. **Code review security**
5. **New service onboarding (main platform story)**

Release safety

What this proves

Bad releases never reach production. Automated checks block unsafe deploys.

Buttons

Block unsafe release

What happens: A deliberately bad deployment is sent to your CI pipeline.

- Behind the scenes: GitHub Actions runs policy checks
- Checks fail on purpose: bad image tag, missing limits, wrong registry
- Client sees: Latest outcome shows checks failed - that is success for this demo
- Show them: Build pipelines tab on GitHub - red X on policy step

Say to client: "Our pipeline stops unsafe releases before they touch the cluster. Bad config never gets deployed."

Approve safe release

What happens: A clean, compliant deployment is sent to the same pipeline.

- Client sees: Checks pass - green pipeline

Say to client: "When everything meets policy, the release is approved automatically. Same gate, opposite result."

Outage response

What this proves

When an app breaks, AI explains why and the platform rolls back automatically.

1. Simulate outage

What happens: Staging app is broken on purpose (bad container image).

- Client sees: Status may flicker; Deployments tab shows Degraded
- Open: http://localhost:8082 - fastapi-staging turns red

Say to client: "We have a live incident in staging - on purpose - so you can see how we respond."

2. Explain with AI

What happens: AI reads cluster signals and summarizes the failure.

- Client sees: Findings in What just happened panel on the right

Say to client: "AI triages the incident in seconds - root cause without digging through logs."

3. Auto-fix app

What happens: GitOps rolls back to the last good version and restarts the app.

- Client sees: App healthy again; click Refresh if dashboard still shows fail
- Note: After heal, Refresh restores the browser tunnel - pod was already fine

Say to client: "Recovery is automated. We restore the last known good deployment - no manual kubectl."

Cloud config guard

What this proves

Git is the contract for cloud settings. We detect when someone changes the cloud outside that process, and we restore the approved state automatically.

The two boxes on screen

- Left - What we want (in Git): approved secure settings
- Right - What's running now: actual live settings
- MATCHES GIT = good | OUT OF SYNC = someone drifted from Git

1. Set secure baseline

One sentence: Apply the approved configuration from Git so live matches the rules.

What happens when you click

- Local cloud sandbox starts (background - client never sees this)
- Terraform apply runs from foundation/terraform/demo4
- Creates bucket enlight-demo: Private + encrypted (AES256)

What client sees

- Latest outcome: Secure baseline set
- Both boxes show Private (secure)
- Badge: MATCHES GIT (green)

Say to client: "Our secure storage rules live in Git. This applies that approved definition. Now Git and the live environment agree."

2. Catch config drift

One sentence: Simulate someone breaking the rules outside Git.

What happens when you click

- Script changes bucket ACL to public-read (like a rogue console edit)
- Terraform plan detects live no longer matches Git

What client sees

- Left: Private (secure) | Right: Public (unsafe)
- Badge: OUT OF SYNC (red)
- Extra cost risk may appear (~$15/mo demo estimate)

Say to client: "Someone changed cloud settings outside our approved process. We caught it immediately - that is drift, and it is a security and cost risk."

3. Fix drift automatically

One sentence: Put live settings back to what Git says - one click.

What happens when you click

- Terraform apply runs again
- Bucket returns to private + encrypted

What client sees

- Latest outcome: Restored to match Git
- Both boxes Private (secure) again
- Badge: MATCHES GIT (green)

Say to client: "We do not fix this by hand in the console. We reconcile from Git automatically."

All three steps as a story

```
Step 1  Git: Private   Live: Private   -> MATCHES GIT
Step 2  Git: Private   Live: Public    -> OUT OF SYNC (drift)
Step 3  Git: Private   Live: Private   -> MATCHES GIT (fixed)
```

Say to client: "Git is the contract. We detect drift and restore it automatically."

Code review security

What this proves

Every change request is scanned before merge. Secrets and unsafe config cannot slip through.

Open risky change request

What happens: Opens a new GitHub pull request with hardcoded secrets + bad S3 settings.

- Client sees: PR opened; security checks fail (red)
- Normal: ONE pull request, TWO check workflows (service CI + compliance bot)

Say to client: "Every PR is scanned. This one fails on purpose - secrets and unsafe config are blocked."

Open safe change request

What happens: Opens a new PR with clean, compliant configuration.

- Client sees: Checks pass (green)

Say to client: "Clean changes pass the same gates. Merge is optional - the checks are the proof."

New service onboarding (main story)

What this proves

A developer can spin up a full production-ready service in minutes - with monitoring, pipelines, and deployment already wired. This is the platform vision.

1. Create new app

What happens: Platform scaffolds a brand-new service (svc-TIMESTAMP each time).

- Bundle includes: Kubernetes manifests, CI workflow, Terraform, ArgoCD app, monitoring
- Client sees: App being built shows new svc- name

Say to client: "From the portal we provision a new service with all platform defaults - not a blank repo, a golden path."

2. Submit for review

What happens: Opens a GitHub PR so the team can review every file.

- Client sees: Review request # opened; CI runs on GitHub

Say to client: "Nothing goes live without a reviewable pull request - full audit trail."

3. Go live

What happens: Registers the app in GitOps and deploys to the cluster.

- Client sees: New app in Deployments at http://localhost:8082
- Tip: Merge PR on GitHub first for the full GitOps story (optional)

Say to client: "One click from approved code to a running service with monitoring already attached."

Guided walkthrough (10 steps)

Use the purple Guided walkthrough card - same demos, scripted order:

1. Health check - verify all status cards green
2. Block unsafe release
3. Approve safe release
4. Simulate outage
5. Explain with AI
6. Auto-fix app
7. Open risky change request
8. Create new app
9. Catch config drift
10. Fix drift automatically

Quick fixes

If you see this...

- Dashboard shows fail but ArgoCD healthy -> Click Refresh on Live Demo
- Cloud guard shows NOT STARTED -> Click Set secure baseline
- Nothing loads -> Run go-live.bat then Refresh
- ArgoCD blank -> Use http://localhost:8082 not 8080

Likely client questions

Why not real AWS?

Local sandbox = $0 today. Same architecture on EKS in production.

What is the main demo?

New service onboarding - everything else proves safety gates around it.

Two GitHub workflows on one PR?

Normal - service CI plus org compliance bot. Both gates on every change.